Runway-Relative Yaw and Lateral Offset from a Single Monocular Camera on an Embedded GPU

Samuel Moyal
Carnegie Mellon University

Abstract

[TODO: write last.]

1 Introduction

On final approach, an aircraft must know its heading relative to the runway axis (yaw) and its cross-track offset from the centerline (lateral offset). GNSS and ILS provide them when available; when GNSS is jammed or the airfield has no ILS, a forward-looking camera is the only sensor that observes the runway itself. We estimate yaw and lateral offset from a single monocular RGB camera, assuming the runway dimensions are known (from a runway database), under two deployment constraints: the full pipeline must run on an NVIDIA Jetson Orin NX 16 GB at more than 10 Hz, and it must hold under adverse weather (fog, rain, night, glare).

Our pipeline segments the runway, fits its four corners, recovers the camera pose from the runway-plane homography, and filters it with a kinematic EKF. [TODO: contributions list once experiments are done.]

2 Related Work

[TODO: vision-based landing; LARD, BARS, RLD; real-time semantic / instance segmentation; corner keypoint regression; data scaling laws; sim-to-real with diffusion.]

3 Problem and Constraints

[TODO: runway frame, camera model, definition of yaw / lateral offset; latency budget (100 ms end-to-end, worst case includes the full-frame fallback of the crop tracker).]

4 Data

Datasets. We use three public runway datasets and one of our own. LARD [2] provides 14,668 synthetic images rendered at 117 airports, each with its four runway corners and the full 6-DoF camera pose (slant distance, lateral and vertical path angles); it is the only public set with pose. BARS [1] (10,255 images, mostly X-Plane) and RLD (12,254 images) provide runway polygons or quadrilaterals but no pose. Our X-Plane 12 set (XP12) contains 118 continuous approaches at 1280×720 px with ground-truth pose for every frame, from 2.8 km to the threshold; it is our sequential pose benchmark. [TODO: LARD real test set: archive incomplete, re-download.]

Uniform audit. Published statistics are not comparable across datasets, so we compute the same descriptors on every image. Geometry comes from the labels: number of runway instances and width of the largest runway once the image is resized to the 640-px network input. Pose statistics come from the metadata where available. For lighting and weather, LARD's `weather` and `night` columns are empty, and a pixel-level haze proxy (dark-channel mean and RMS contrast) proved unreliable: on inspection it mostly selected bright, low-contrast desert ground rather than fog. We therefore tag each image zero-shot with CLIP ViT-L/14 [3]: each attribute (night, dusk, fog, rain on the windshield, snow, sun glare, overcast) is a pair of positive and negative prompts, and its score is the softmax probability of the positive prompt (0.5 is chance). Thresholds are fixed per attribute and checked by hand on contact sheets of the tagged images. [TODO: thresholds and resulting percentages, **??**, **??**.]

Findings so far. Small runways dominate: at a 640-px input, the median runway is 14 px wide in LARD and 20 px in XP12, and 54 % of LARD images (30 % of XP12 frames) show a runway narrower than 16 px. LARD spans 0.15–4.3 km and lateral path angles of $\pm 4.4°$ (5th–95th percentile); XP12 spans 0.15–2.8 km, -1.7 to $2.2°$, with relative yaw within $\pm 9°$. Neither set contains night images (mean luminance never below 60). [TODO: BARS / RLD audit; weather percentages from CLIP; airport overlap between BARS splits.]

5 Method

[TODO: segmentation → quad → homography (5 Gauss–Newton iterations) → EKF; closed-loop crop at $t-1$.]

Table 1: YOLO-seg latency (ms) on an NVIDIA L4, TensorRT FP16, batch 1, 1280 × 720 input. Inf.: network only; total: pre-processing, inference and mask post-processing (mean of 200 runs).

	640		960		1280	
Model	inf.	total	inf.	total	inf.	total
v8n	0.75	2.39	1.20	4.38	1.91	3.95
11n	0.84	2.50	1.32	4.55	2.05	4.55
26n	0.89	2.62	1.36	4.68	2.13	4.45
v8s	1.22	2.86	2.26	5.35	4.21	6.33
11s	1.26	2.89	2.34	5.44	4.26	6.40
26s	1.37	2.97	2.50	5.58	4.74	6.88
v8m	2.74	4.35	5.51	8.60	11.49	13.80
11m	2.75	4.35	5.77	8.88	12.11	14.39
26m	2.93	4.60	6.33	9.49	13.06	15.48

6 Experiments

Evaluation protocol. We optimise sequentially: first the segmentation model with the post-processing frozen, then the post-processing with the segmentation fixed, and finally re-rank the best segmentation models with the tuned post-processing to check that the ranking holds. Segmentation is reported with mask IoU, boundary IoU and corner error; every model is also run through a frozen reference post-processing so that its pose error is reported alongside.

6.1 Segmentation models: accuracy vs. latency

Latency. Latency depends on the architecture and input size only, so we measure it once with COCO-pretrained weights (Table 1). At equal size, the three YOLO generations (v8, 11, 26) are within 10 % of each other; model size and input resolution dominate. For the nano models, pre- and post-processing (letterboxing the 1280 × 720 frame, decoding and upsampling masks) cost more than the network itself, so the end-to-end time is not proportional to the FLOPs. [TODO: convert to Jetson Orin NX: calibrate the L4→Orin NX factor with YOLOv8s detection (7.94 ms FP16 on Orin NX 16 GB reported by Ultralytics); check the 1280 vs. 960 anomaly for nano models.]

Training protocol. All models are trained on the union of BARS and RLD restricted to the runway class (17,858 training, 2,001 validation and 2,650 test images). BARS keeps its official split; RLD has no airport metadata and its file numbering follows the source videos, so we split it by blocks of 200 consecutive images (80/10/10) to keep near-duplicate frames on the same side. Every run uses the Ultralytics defaults (SGD, 40 epochs at most, early stopping after 10 epochs without improvement, mosaic, random scale ±50 %, horizontal flip) and evaluates at its training resolution.

Search strategy. A full grid over three generations, three sizes and three input resolutions would take 27 training runs. We instead optimise one axis at a time: generation at nano size and 640 px, then model size with the best generation, then input resolution with the best model, and finally one cross-check of the size–resolution interaction at matched latency (e.g. nano at 1280 px vs. small at 960 px), the only coupling we expect to matter.

Is the resolution comparison biased by augmentation? Mosaic and random scaling are defined relative to the training resolution: each image is resized so its long side equals the input size before being tiled and rescaled by a factor in $[0.5, 1.5]$. Training and test object sizes therefore scale together with the input resolution, and every model sees a training distribution matched to its own test resolution; the comparison is fair in that sense. Two caveats remain. First, the 0.5× end of the scale augmentation shrinks already-small runways below the stride of the finest feature map (8 px) at 640 px, wasting part of the training signal precisely on the far range; this penalises low resolutions more than high ones, which is a property of the recipe rather than a bias, but we report it. Second, mosaic trains on square crops while inference uses rectangular letterboxed frames; this is identical across resolutions. Native resolutions are 1920 × 1080 (BARS) and mostly 1280 × 720 (RLD), so 1280 px adds real detail for BARS but is close to native for RLD; upsampling beyond native adds no information but still spreads small objects over more cells of the stride-8 grid.

Training budget. For YOLO11n at 640 px, validation mask mAP50–95 rises from 0.52 after one epoch to 0.65 at epoch 11 and stays within ±0.01 up to epoch 16 (150 s per epoch on an L4). We therefore train the remaining configurations for 25 epochs; YOLO11n is trained both for 40 and 25 epochs to measure what the shorter budget costs. [TODO: final 40 vs. 25 numbers.]

Apparent runway size, not resolution. Distance, field of view and input resolution act on the detector only through the apparent size of the runway at the network input. For a runway of width W_r seen at slant distance d by a camera of horizontal field of view ϕ whose image is resized to W_{in} pixels wide, the apparent width is

$$s = \frac{W_{\text{in}}\, W_r}{2\, d \tan(\phi/2)}. \tag{1}$$

We therefore report accuracy as a function of s rather than of d or W_{in} separately. If the curves for different input resolutions collapse onto one curve of s, the model has a resolution-independent operating band $[s_{\min}, s_{\max}]$, and Eq. (1) gives the input width needed at each distance; with $W_r = 45$ m, $\phi = 65°$ and $s_{\min} = 20$ px, a runway at

Table 2: Pose error by mask source, homography + EKF, 13 XP12 approaches.

Mask source	meas.	lat MAE	lat p95	yaw MAE
Full frame	68.5 %	3.37 m	10.8 m	0.18°
Crop $t-1$	69.1 %	**3.08 m**	11.0 m	**0.16°**
Conf. 0.1	81.6 %	8.45 m	18.4 m	0.43°
BoT-SORT	81.6 %	8.45 m	18.4 m	0.43°
Crop + conf. 0.1	82.1 %	8.68 m	18.1 m	0.51°

3 km requires $W_{\text{in}} \approx 1700$ px. This is the quantitative argument for the crop tracker of Section 6.4: rather than raising the resolution of the whole frame, it keeps the runway inside the operating band at constant cost. [TODO: curves of IoU and corner error vs. s for the three resolutions; estimate $s_{\min}$, $s_{\max}$.]

Accuracy. [TODO: E1 results: test mask mAP on BARS and RLD, then IoU / corner error by distance on LARD and XP12.]

6.2 Other model families

[TODO: E2: DeepLabV3+ / LR-ASPP MobileNetV3, SegFormer-B0, PIDNet, RTMDet-Ins, 4-corner keypoint regression.]

6.3 Data scaling

[TODO: E3: error$(N) = aN^{-b} + c$ for 3 models.]

6.4 Temporal modules on top of the segmenter

We compare four ways of producing the runway mask on 13 X-Plane 12 approaches (2,801 frames, 2.8 km to threshold), with the same YOLOv8n-seg weights and no retraining: (i) full frame at 640 px (baseline); (ii) a closed-loop crop: a 16:9 window four times the size of the runway found at $t-1$ (at least 256 px), resized to 640 px, falling back to the full frame when nothing is found; (iii) a lower confidence threshold (0.1, the default of Ultralytics tracking); (iv) BoT-SORT via `model.track`, keeping the raw masks of the followed track. Pose is estimated with homography (5 iterations) and the kinematic EKF in all cases (every 4th frame, 1,404 frames).

The crop raises the share of frames with mask IoU > 0.5 from 46.3 % to 59.1 % overall and from 34 % to 73 % in the last 500 m, and lowers lateral error at 1–1.5 km from 2.1 to 1.4 m (Table 2). It does not help beyond 2 km, where the problem is the first detection, not its refinement. Lowering the threshold increases availability at 2–3 km from 29 % to 51 % but those masks are poor: lateral error there grows from 11 to 36 m once they enter the EKF. BoT-SORT is identical to the 0.1 threshold to the third decimal: with a single runway there is nothing to associate, and Ultralytics returns the raw masks of matched detections and nothing for lost tracks, so it cannot fill gaps that the EKF already coasts through. [TODO: re-run on all 86 approaches with the final segmenter; use low-confidence detections only to place the re-acquisition crop.]

7 Limitations and Future Work

[TODO:]

References

[1] Wenhui Chen et al. BARS: A benchmark for airport runway segmentation. arXiv preprint arXiv:2210.12922, 2022.

[2] Mélanie Ducoffe, Maxime Carrere, Léo Féliers, Adrien Gauffriau, Vincent Mussot, Claire Pagetti, and Thierry Sammour. LARD – landing approach runway detection – dataset for vision based landing. In *arXiv preprint arXiv:2304.09938*, 2023.

[3] Alec Radford, Jong Wook Kim, Chris Hallacy, et al. Learning transferable visual models from natural language supervision. In *ICML*, 2021.